

List Indexing in Python: Accessing Elements Efficiently

In Python, lists allow you to access individual elements using their index. Indexing starts at `0`, meaning the first element in a list is at index `0`, the second element is at index `1`, and so on. This is called **zero indexing**. To access an element, use square brackets with the index of the desired item.

```
cities = ["Rome", "London", "New York", "Brasilia", "Kioto"]

# Getting the first element
print(cities[0])

# Getting the third element
print(cities[2])
```

In the next example, the index `0` returns the first element, 'Rome', while the index `2` returns the third element, which is 'New York'. Remember that the index corresponds to the position minus one ($n - 1$).

Negative Indexing

Python also supports **negative indexing**: this allows you to access elements from the end of the list. Here, `-1` represents the last item, `-2` represents the second last item, and so on. Negative indexing can be very useful when you want to work with a list from the end without knowing its length.

```
cities = ["Rome", "London", "New York", "Brasilia", "Kioto"]

# Getting the last element
print(cities[-1])

# Getting the fourth element
print(cities[3], cities[-2])
```

Explanation: 1. `-1` retrieves the last element 'Kioto'; 2. `3` and `-2` both refer to the fourth element 'Brasilia', one using positive indexing and the other using negative indexing. This demonstrates how indexing wraps around the list.

Positive and negative indexing add versatility to the access of elements from lists, either from the beginning or the end.

Indexing in Nested Lists

Accessing elements in a nested list requires multiple indices: the first index selects the sublist, and the second index accesses the specific item within that sublist.

```
cities = [  
    ["Paris", "France", 2000],  
    ["Tokyo", "Japan", 3000],  
    ["New York", "USA", 2500]  
]  
  
# Accessing the first sublist  
print(cities[0]) # Output: ['Paris', 'France', 2000]  
  
# Accessing the city name in the second sublist  
print(cities[1][0]) # Output: Tokyo  
  
# Accessing the cost of the trip in the third sublist  
print(cities[2][2]) # Output: 2500
```

Applications of nested list indexing include but are not limited to, structured data such as **spreadsheets**, **matrices**, or **databases**. Practical examples could be accessing rows and columns in a 2D matrix, retrieving details from lists of employee records, or extracting specific information, such as the city names or costs from travel itineraries or nested JSON-like structures.